

```
In [1]: from tensorflow import keras
```

```
In [2]: pip install stdlib-list
```

```
Collecting stdlib-list
  Using cached stdlib_list-0.11.1-py3-none-any.whl.metadata (3.3 kB)
Using cached stdlib_list-0.11.1-py3-none-any.whl (83 kB)
Installing collected packages: stdlib-list
Successfully installed stdlib-list-0.11.1
Note: you may need to restart the kernel to use updated packages.
```

```
In [4]: pip show tensorflow
```

```
Name: tensorflow
Version: 2.13.0
Summary: TensorFlow is an open source machine learning framework for everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: c:\users\lenovo\project\code\dl\myenv\lib\site-packages
Requires: tensorflow-intel
Required-by:
Note: you may need to restart the kernel to use updated packages.
```

```
In [5]: pip show keras
```

```
Name: kerasNote: you may need to restart the kernel to use updated packages.

Version: 2.13.1
Summary: Deep learning for humans.
Home-page: https://keras.io/
Author: Keras team
Author-email: keras-users@googlegroups.com
License: Apache 2.0
Location: c:\users\lenovo\project\code\dl\myenv\lib\site-packages
Requires:
Required-by: tensorflow-intel
```

```
In [6]: from tensorflow.keras.preprocessing.image import ImageDataGenerator, array_to_img,

datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

```
In [7]: import tensorflow.keras as keras
```

```
In [8]: img = load_img(r'C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download.jpeg
```

In [9]: `img`

Out[9]:



```
In [10]: x = img_to_array(img) # this is a Numpy array with shape (3, 150, 150)
x = x.reshape((1,) + x.shape) # this is a Numpy array with shape (1, 3, 150,150)
# the .flow() command below generates batches of randomly transformed images
# and saves the results to the `preview/` directory
i = 0
for batch in datagen.flow(x, batch_size=1,
    save_to_dir=r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\data_augum",
    i += 1
    if i > 30:
        break # otherwise the generator would loop indefinitely
```

```
In [11]: from tensorflow.keras.utils import image_dataset_from_directory
from keras.utils import array_to_img ,img_to_array, load_img
datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='reflect')
img = load_img(r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpe
2
```

Out[11]: 2

In [12]: `img`

Out[12]:



```
In [13]: x = img_to_array(img) # this is a Numpy array with shape (3, 150, 150)
x = x.reshape((1,) + x.shape) # this is a Numpy array with shape (1, 3, 150,150)
# the .flow() command below generates batches of randomly transformed images
# and saves the results to the `preview/` directory
i = 0
for batch in datagen.flow(x, batch_size=1,
    save_to_dir=r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\data_augum",
    i += 1
    if i > 30:
        break # otherwise the generator would loop indefinitely
```

Resnet50

```
In [14]: keras.applications.ResNet50(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax",
)
```

Out[14]: <keras.src.engine.functional.Functional at 0x2389b64b580>

1. Basic Predictions of the Image

2. ResNet50 Model

```
In [15]: from keras.utils import array_to_img, img_to_array, load_img
img = load_img(r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpe
img
```

Out[15]:



```
In [16]: import numpy as np
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
# Load the ResNet-50 model pre-trained on ImageNet data
model = ResNet50(weights='imagenet')
# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
```

```

img_array = image.img_to_array(img)

img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]

print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

```

1/1 [=====] - 3s 3s/step

Predictions:

```

1: daisy (1.00)
2: cabbage_butterfly (0.00)
3: sulphur_butterfly (0.00)

```

Top Prediction Class Index: 985

ResNet50V2 Model

```

In [19]: import numpy as np
from tensorflow.keras.applications.resnet_v2 import ResNet50V2, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the ResNet50V2 model pre-trained on ImageNet data
model = ResNet50V2(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #

img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make Predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predictions:")

for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction

```

```
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

1/1 [=====] - 2s 2s/step

Predictions:

```
1: daisy (1.00)
2: bee (0.00)
3: vase (0.00)
```

Top Prediction Class Index: 985

VGG16 Model

```
In [20]: from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the VGG16 model pre-trained on ImageNet data
model = VGG16(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]

print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels.h5

553467096/553467096 [=====] - 203s 0us/step

1/1 [=====] - 4s 4s/step

Predictions:

```
1: monarch (0.92)
2: admiral (0.07)
3: lycaenid (0.01)
```

Top Prediction Class Index: 323

```
In [21]: from keras.utils import array_to_img, img_to_array, load_img
img = load_img(r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpe
img
```

Out[21]:



```
In [22]: import numpy as np
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the VGG16 model pre-trained on ImageNet data
model = VGG16(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

WARNING:tensorflow:5 out of the last 5 calls to <function Model.make_predict_function at 0x00000238AE911BD0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

1/1 [=====] - 4s 4s/step

Predictions:

1: daisy (0.43)
2: ant (0.12)
3: cabbage_butterfly (0.09)

Top Prediction Class Index: 985

VGG19 Model

```
In [43]: from keras.utils import array_to_img, img_to_array, load_img
img = load_img(r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download2.jpe
img
```

Out[43]:



```
In [25]: import numpy as np
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input, decode_pred
from tensorflow.keras.preprocessing import image

# Load the VGG19 model pre-trained on ImageNet data
model = VGG19(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images.jpg" # rep
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19_weights_tf_dim_ordering_tf_kernels.h5
574710816/574710816 [=====] - 169s 0us/step
WARNING:tensorflow:6 out of the last 6 calls to <function Model.make_predict_function.<locals>.predict_function at 0x00000238AE910A60> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.
1/1 [=====] - 3s 3s/step
Predictions:
1: unicycle (0.10)
2: cock (0.07)
3: umbrella (0.04)
Top Prediction Class Index: 880

```
In [26]: import numpy as np
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the VGG19 model pre-trained on ImageNet data
model = VGG19(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images.jpg" # replace with your image path
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=3)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```


1/1 [=====] - 4s 4s/step

Predictions:

1: unicycle (0.10)
2: cock (0.07)
3: umbrella (0.04)
4: mountain_bike (0.03)
5: bathing_cap (0.03)
6: gasmask (0.02)
7: crash_helmet (0.02)
8: standard_poodle (0.02)

Top Prediction Class Index: 880

```
In [27]: import numpy as np
from tensorflow.keras.applications.resnet_v2 import ResNet50V2, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the ResNet50V2 model pre-trained on ImageNet data
model = ResNet50V2(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=10)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")
```

1/1 [=====] - 6s 6s/step

Predictions:

1: daisy (1.00)
2: bee (0.00)
3: vase (0.00)
4: ant (0.00)
5: sea_anemone (0.00)
6: sulphur_butterfly (0.00)
7: pot (0.00)
8: sulphur-crested_cockatoo (0.00)
9: binder (0.00)
10: fly (0.00)

Top Prediction Class Index: 985

Predictions for dog image using Keras Api Applications.....

```
In [28]: from keras.utils import array_to_img, img_to_array, load_img
img = load_img(r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images3.jpeg")
img
```

Out[28]:



```
In [30]: import numpy as np
import time
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the ResNet-50 model pre-trained on ImageNet data
model = ResNet50(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images3.jpeg" # r
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()

# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
```

```

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

1/1 [=====] - 5s 5s/step

Predictions:

```

1: groenendael (0.21)
2: German_shepherd (0.14)
3: briard (0.09)
4: golden_retriever (0.07)
5: collie (0.06)

```

Top Prediction Class Index: 224

Inference Time: 5092.86 ms

Size (MB): 97.80 MB

Parameters: 25636712

Depth: 177

```

In [33]: import numpy as np
from tensorflow.keras.applications.resnet_v2 import ResNet50V2, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the ResNet50V2 model pre-trained on ImageNet data
model = ResNet50V2(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download2.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()

```

```

# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

1/1 [=====] - 5s 5s/step

Predictions:

```

1: golden_retriever (0.99)
2: kuvasz (0.01)
3: Great_Pyrenees (0.00)
4: Labrador_retriever (0.00)
5: English_setter (0.00)

```

Top Prediction Class Index: 207

Inference Time: 4797.12 ms

Size (MB): 97.71 MB

Parameters: 25613800

Depth: 192

```

In [34]: import numpy as np
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
import time

# Load the VGG19 model pre-trained on ImageNet data
model = VGG19(weights='imagenet')
# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download2.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

```

```

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

1/1 [=====] - 6s 6s/step

Predictions:

1: Great_Pyrenees (0.50)
 2: golden_retriever (0.25)
 3: kuvasz (0.17)
 4: Tibetan_mastiff (0.03)
 5: otterhound (0.01)

Top Prediction Class Index: 257

Inference Time: 8480.38 ms

Size (MB): 548.05 MB

Parameters: 143667240

Depth: 26

```

In [35]: import numpy as np
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image

# Load the VGG16 model pre-trained on ImageNet data
model = VGG16(weights='imagenet')
# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download2.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step

```

```

inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

1/1 [=====] - 4s 4s/step

Predictions:

```

1: otterhound (0.38)
2: golden_retriever (0.24)
3: Afghan_hound (0.20)
4: Great_Pyrenees (0.05)
5: kuvasz (0.04)

```

Top Prediction Class Index: 175

Inference Time: 4942.33 ms

Size (MB): 527.79 MB

Parameters: 138357544

Depth: 23

```

In [36]: import numpy as np
from tensorflow.keras.applications import Xception
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.xception import preprocess_input, decode_predictions

# Load the Xception model pre-trained on ImageNet data
model = Xception(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images3.jpeg" # r
img = image.load_img(img_path, target_size=(299, 299)) # Xception requires input sh
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction

```

```

top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/xception/xception_weights_tf_dim_ordering_tf_kernels.h5

91884032/91884032 [=====] - 27s 0us/step

1/1 [=====] - 10s 10s/step

Predictions:

1: German_shepherd (0.24)
 2: Leonberg (0.05)
 3: malinois (0.05)
 4: Pembroke (0.04)
 5: chow (0.03)

Top Prediction Class Index: 235

Inference Time: 10115.31 ms

Size (MB): 87.40 MB

Parameters: 22910480

Depth: 134

```

In [37]: import numpy as np
         from tensorflow.keras.applications import InceptionV3
         from tensorflow.keras.preprocessing import image
         from tensorflow.keras.applications.inception_v3 import preprocess_input, decode_predictions

         # Load the InceptionV3 model pre-trained on ImageNet data
         model = InceptionV3(weights='imagenet')

         # Load and preprocess the input image
         img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images3.jpeg" # r
         img = image.load_img(img_path, target_size=(299, 299)) # Xception requires input sh
         img_array = image.img_to_array(img)
         img_array = np.expand_dims(img_array, axis=0)
         img_array = preprocess_input(img_array)

         # Measure inference time
         start_time = time.time()
         predictions = model.predict(img_array)
         end_time = time.time()

```

```

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_v3/inception_v3_weights_tf_dim_ordering_tf_kernels.h5
96112376/96112376 [=====] - 29s 0us/step
1/1 [=====] - 12s 12s/step
Predictions:
1: golden_retriever (0.28)
2: chow (0.14)
3: German_shepherd (0.07)
4: malinois (0.04)
5: dingo (0.03)

Top Prediction Class Index: 207
Inference Time: 12341.41 ms
Size (MB): 90.99 MB
Parameters: 23851784
Depth: 313

```

```

In [38]: import numpy as np
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_pred

# Load the MobileNetV2 model pre-trained on ImageNet data
model = MobileNetV2(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\images3.jpeg" # r
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)

```



```

img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")
# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")
# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5

14536120/14536120 [=====] - 5s 0us/step

1/1 [=====] - 7s 7s/step

Predictions:

```

1: German_shepherd (0.25)
2: malinois (0.13)
3: groenendael (0.06)
4: collie (0.05)
5: red_wolf (0.04)

```

Top Prediction Class Index: 235

Inference Time: 7555.03 ms

Size (MB): 13.50 MB

Parameters: 3538984

Depth: 156

```

In [39]: import numpy as np
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.applications.densenet import preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
# Load the DenseNet121 model pre-trained on ImageNet data
model = DenseNet121(weights='imagenet')

# Load and preprocess the input image

```

```

img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_ordering_tf_kernels.h5

33188688/33188688 [=====] - 10s 0us/step

1/1 [=====] - 21s 21s/step

Predictions:

```

1: daisy (0.93)
2: ant (0.01)
3: lacewing (0.01)
4: bee (0.01)
5: cabbage_butterfly (0.00)

```

Top Prediction Class Index: 985

Inference Time: 22253.95 ms

Size (MB): 30.76 MB

Parameters: 8062504

Depth: 429

In [40]: `import numpy as np`
`from tensorflow.keras.applications import NASNetMobile`

```

from tensorflow.keras.applications.nasnet import preprocess_input, decode_prediction
from tensorflow.keras.preprocessing import image

# Load the NASNetMobile model pre-trained on ImageNet data
model = NASNetMobile(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0
print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/nasnet/NASNet-mobile.h5>
 24227760/24227760 [=====] - 8s 0us/step
 1/1 [=====] - 30s 30s/step
 Predictions:
 1: daisy (0.90)
 2: bee (0.00)
 3: ant (0.00)
 4: pot (0.00)
 5: sulphur_butterfly (0.00)

 Top Prediction Class Index: 985
 Inference Time: 30321.93 ms
 Size (MB): 20.32 MB
 Parameters: 5326716
 Depth: 771

```

In [41]: import numpy as np
         from tensorflow.keras.applications import NASNetLarge
         from tensorflow.keras.preprocessing import image
         from tensorflow.keras.applications.nasnet import preprocess_input, decode_prediction

         # Load the NASNetLarge model pre-trained on ImageNet data
         model = NASNetLarge(weights='imagenet')

         # Load and preprocess the input image
         img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download1.jpeg" #
         img = image.load_img(img_path, target_size=(331, 331)) # NASNetLarge requires input
         img_array = image.img_to_array(img)
         img_array = np.expand_dims(img_array, axis=0)
         img_array = preprocess_input(img_array)

         # Measure inference time
         start_time = time.time()
         predictions = model.predict(img_array)
         end_time = time.time()

         # Decode and print the top-3 predicted classes
         decoded_predictions = decode_predictions(predictions, top=5)[0]
         print("Predictions:")
         for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
             print(f"{i + 1}: {label} ({score:.2f})")

         # Optionally, you can obtain the class index for the top prediction
         top_class_index = np.argmax(predictions[0])
         print(f"\nTop Prediction Class Index: {top_class_index}")

         # Calculate and print the inference time per step
         inference_time_ms = (end_time - start_time) * 1000.0
         print(f"Inference Time: {inference_time_ms:.2f} ms")

         # Model summary provides information about parameters and layers
         #model.summary()

         # Get the size of the model in megabytes
         model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
         print(f"Size (MB): {model_size_MB:.2f} MB")
  
```

```

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/nasnet/NASNet-large.h5>

359748576/359748576 [=====] - 116s 0us/step
 1/1 [=====] - 30s 30s/step

Predictions:

1: daisy (0.93)
 2: bee (0.00)
 3: ant (0.00)
 4: pot (0.00)
 5: vase (0.00)

Top Prediction Class Index: 985

Inference Time: 30426.50 ms

Size (MB): 339.32 MB

Parameters: 88949818

Depth: 1041

```

In [42]: import numpy as np
from tensorflow.keras.applications import EfficientNetV2B0
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.efficientnet_v2 import preprocess_input, decode_

# Load the EfficientNetV2B0 model pre-trained on ImageNet data
model = EfficientNetV2B0(weights='imagenet')

# Load and preprocess the input image
img_path = r"C:\Users\Lenovo\project\Code\DL\Keras_Pretand_models\download2.jpeg" #
img = image.load_img(img_path, target_size=(224, 224)) # NASNetLarge requires input
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Measure inference time
start_time = time.time()
predictions = model.predict(img_array)
end_time = time.time()

# Decode and print the top-3 predicted classes
decoded_predictions = decode_predictions(predictions, top=5)[0]
print("Predictions:")
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i + 1}: {label} ({score:.2f})")

# Optionally, you can obtain the class index for the top prediction
top_class_index = np.argmax(predictions[0])
print(f"\nTop Prediction Class Index: {top_class_index}")

# Calculate and print the inference time per step
inference_time_ms = (end_time - start_time) * 1000.0

```

```

print(f"Inference Time: {inference_time_ms:.2f} ms")

# Model summary provides information about parameters and layers
#model.summary()
# Get the size of the model in megabytes
model_size_MB = model.count_params() * 4 / (1024 ** 2) # 4 bytes for float32
print(f"Size (MB): {model_size_MB:.2f} MB")

# Get the number of parameters and depth from the model's layers
num_parameters = model.count_params()
model_depth = len(model.layers)
print(f"Parameters: {num_parameters}")
print(f"Depth: {model_depth}")

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/efficientnet_v2/efficientnetv2-b0.h5

29403144/29403144 [=====] - 16s 1us/step

1/1 [=====] - 6s 6s/step

Predictions:

1: golden_retriever (0.49)

2: kuvasz (0.23)

3: Great_Pyrenees (0.12)

4: Labrador_retriever (0.02)

5: clumber (0.01)

Top Prediction Class Index: 207

Inference Time: 5739.47 ms

Size (MB): 27.47 MB

Parameters: 7200312

Depth: 273

In []: